

**BLOOD DONATION CAMPAIGN EFFECTIVENESS ANALYTICS**

**MINI PROJECT REPORT**

*Submitted by*

**ABARNAA  
MADHUMITHA R  
MONISHA K**

**REGISTER NO: 23UCB001  
REGISTER NO: 23UCB034  
REGISTER NO: 23UCB039**

*Under the Guidance of*

**Mrs. E. THAMIZHARASI**

*Assistant Professor*

*in partial fulfilment for the award of the degree of*

**BACHELOR OF TECHNOLOGY**

**in**

**DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS**



**SRI MANAKULA VINAYAGAR ENGINEERING COLLEGE  
(AN AUTONOMOUS INSTITUTION)**

**MADAGADIPET, PUDUCHERRY-605107**

**APRIL 2026**

## **DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS**

### **BONAFIDE CERTIFICATE**

This is to certify that the project work entitled "**BLOOD DONATION CAMPAIGN EFFECTIVENESS ANALYTICS**" is a Bonafide work done by ABARNAA [23UCB001], MADHUMITHA R [23UCB034], MONISHA K [23UCB039] in partial fulfillment of the requirement, for the award of B.Tech Degree in Computer Science and Business Systems by Pondicherry University during the academic year 2025 – 2026.

#### **PROJECT GUIDE**

**Mrs. E. THAMIZHARASI,**  
Assistant Professor, Department of CSEBS,  
Sri Manakula Vinayagar Engg. College,  
Madagadipet, Puducherry - 605 107.

#### **PROJECT COORDINATOR**

#### **HEAD OF THE DEPARTMENT**

**Dr. M. AUXILIA,**  
Professor & Head, Department of CSEBS,  
Sri Manakula Vinayagar Engg. College,  
Madagadipet, Puducherry - 605 107.

Submitted for the End Semester Practical Examination held on \_\_\_\_\_

#### **INTERNAL EXAMINER**

#### **EXTERNAL EXAMINER**

## **PROBLEM STATEMENT**

### **AWS Academy Cloud Foundations**

Blood banks conduct multiple donation campaigns through channels such as WhatsApp, SMS, and email, but lack a centralised system to store and analyse campaign data. Campaign logs, donor responses, and actual donation records are scattered across systems, making it difficult to derive actionable insights. Without a unified analytics layer, blood bank coordinators must manually compile reports, increasing the risk of error and delay. This project addresses the need for a cloud-based data lake using Amazon S3 to store campaign send logs, donor responses, and blood donation records. Amazon Athena is used to query campaign performance data using serverless SQL, while AWS Lambda enables serverless processing of campaign analytics — all core concepts covered in the AWS Academy Cloud Foundations course.

### **AWS Academy Generative AI Foundations**

Blood bank coordinators require an easy way to understand campaign performance using natural language queries such as 'Which campaign performed best?' or 'Which channel gives highest donation conversion?' without writing SQL queries or reading raw data files. Traditional analytics systems cannot provide this capability without integrating large language models (LLMs). This project designs a Generative AI-based analytics assistant using Retrieval-Augmented Generation (RAG) principles that interprets user queries, retrieves campaign data from Amazon S3, and generates meaningful insights without requiring direct SQL knowledge. The assistant applies intent classification, data grounding, and Chain-of-Thought prompting to deliver natural language responses.

## **AWS Academy Machine Learning Foundations**

Predicting donor behaviour is critical to improving campaign effectiveness. Blood banks need to identify which donors are likely to respond to campaigns and proceed to donate, enabling targeted outreach rather than broad mass campaigns. This project implements a machine learning model using Amazon SageMaker to predict donor response probability and optimise campaign targeting. Additionally, an advanced donor lapse prediction model is developed to identify donors at risk of not donating within the next six months, enabling proactive engagement strategies. The ML workflow covers problem framing, feature engineering, model training, evaluation, and serverless inference deployment.

## **AWS Academy Learner Lab**

Deploying a complete cloud-based analytics pipeline requires integration of multiple AWS services in a real environment. This project uses the AWS Academy Learner Lab to implement and test services such as Amazon S3, AWS Glue, Amazon EMR (Apache Spark), Amazon Athena, Amazon QuickSight, Amazon SageMaker, Amazon DynamoDB, and Amazon SES within a controlled sandbox. The system is designed for reproducibility in the face of session resets and follows secure cloud deployment practices including SSE-S3 encryption and IAM LabRole-based identity control.

## TABLE OF CONTENTS

| Sl. NO   | PARTICULARS                                                 | PAGE NO   |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>MODULE – 1: AWS Academy Cloud Foundations</b>            | <b>1</b>  |
| 1.1      | Procedure for Module – 1                                    | 1         |
| 1.2      | Screenshots for Module – 1                                  | 7         |
| 1.3      | Activity Diagram – 1                                        | 9         |
| 1.4      | Result of Module – 1                                        | 11        |
| <b>2</b> | <b>MODULE – 2: AWS Academy Generative AI Foundations</b>    | <b>13</b> |
| 2.1      | Procedure for Module – 2                                    | 13        |
| 2.2      | Screenshots for Module – 2                                  | 18        |
| 2.3      | Activity Diagram – 2                                        | 20        |
| 2.4      | Result of Module – 2                                        | 22        |
| <b>3</b> | <b>MODULE – 3: AWS Academy Machine Learning Foundations</b> | <b>24</b> |
| 3.1      | Procedure for Module – 3                                    | 24        |
| 3.2      | Screenshots for Module – 3                                  | 30        |
| 3.3      | Activity Diagram – 3                                        | 32        |
| 3.4      | Result of Module – 3                                        | 34        |
| <b>4</b> | <b>MODULE – 4: AWS Academy Learner Lab</b>                  | <b>36</b> |
| 4.1      | Procedure for Module – 4                                    | 36        |
| 4.2      | Screenshots for Module – 4                                  | 42        |
| 4.3      | Activity Diagram – 4                                        | 44        |
| 4.4      | Result of Module – 4                                        | 45        |
|          | <b>CONCLUSION</b>                                           | <b>47</b> |
|          | <b>REFERENCES</b>                                           | <b>50</b> |

# **MODULE – 1**

## **AWS Academy Cloud Foundations**

### **1.1 Procedure for Module – 1**

The AWS Academy Cloud Foundations course covers the fundamentals of cloud computing and core AWS services central to this project: Amazon S3, AWS Glue, Amazon Athena, AWS Lambda, and Amazon SES. This module documents the complete implementation of all cloud infrastructure components — from synthetic campaign dataset generation and S3 data lake setup through Glue ETL, Athena SQL analytics, Lambda processing, and SES-based automated report delivery.

#### **Overview**

The Cloud Foundations curriculum teaches AWS global infrastructure, the shared responsibility model, core storage and compute services, ETL data transformation, serverless architecture patterns, and email notification services. All of these concepts were directly applied in building the Blood Donation Campaign Effectiveness Analytics System. The system follows a five-tier architecture: (1) Data storage via Amazon S3, (2) ETL processing via AWS Glue, (3) SQL analytics via Amazon Athena, (4) Serverless processing via AWS Lambda, and (5) Report delivery via Amazon SES — with no server provisioning at any tier.

#### **Understanding the Core AWS Services**

Before implementation, the team studied the following core services from the Cloud Foundations course:

- Amazon S3 (Simple Storage Service): Scalable object storage for any type of data, organised in buckets and folders. Provides 99.999999999% (eleven nines) durability. Used as the centralised data lake for campaign send logs, donor responses, and blood donation records. Three folders segregate raw data, processed data, and query results.
- AWS Glue: Fully managed ETL (Extract, Transform, Load) service. Glue Crawlers automatically discover schema from raw S3 CSV files and populate the Glue Data Catalog. Glue ETL jobs transform raw campaign logs into a unified campaign-donor-donation funnel view used for conversion analysis.

- Amazon Athena: Serverless interactive SQL query service that analyses data directly in S3 via the Glue Data Catalog. No ETL pipelines or database servers required for SQL access. Uses Presto-based SQL engine with a pay-per-query pricing model.
- AWS Lambda: Serverless compute service that executes code in response to events (API calls, S3 events, schedules). Supports Python 3.12. No server management required. Used for campaign analytics processing and triggering SES email report delivery.
- Amazon SES (Simple Email Service): Managed email-sending service used to deliver automated campaign performance summary reports to blood bank coordinators after each analytics pipeline run.
- IAM (Identity and Access Management): Controls who can do what with AWS services. LabRole is the pre-configured IAM role available in the AWS Academy sandbox — grants Lambda permissions for S3 read/write, Glue access, Athena queries, SES sending, and CloudWatch logging.

## Step 1: Dataset Design and Generation

A synthetic but realistic blood donation campaign dataset was designed following the data pipeline principles from the Cloud Foundations course. The dataset simulates twelve months (January–December 2023) of multi-channel donation campaigns, as shown in the following table:

| File Name            | Records  | Key Columns                                                                                    | Purpose in System                                         |
|----------------------|----------|------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| campaign_logs.csv    | 800 rows | campaign_id, channel (WhatsApp/SMS/Email), send_date, donor_id, message_status                 | Campaign send records — denominator for send rate metrics |
| donor_responses.csv  | 520 rows | response_id, campaign_id, donor_id, response_type (clicked/pledged/ignored), response_date     | Donor engagement records — funnel step 2                  |
| donation_records.csv | 310 rows | donation_id, donor_id, campaign_id, donation_date, blood_group, units_donated, donation_center | Actual donations — final funnel conversion                |

Dataset parameters include 20 unique campaign IDs (CAMP001–CAMP020), 300 unique donor IDs (DONOR001–DONOR300), 3 campaign channels (WhatsApp: 40%, SMS: 35%, Email: 25%), overall response rate of 65%, and overall donation conversion from response of 59.6%. WhatsApp was intentionally designed with the highest conversion rate (72%) to produce meaningful channel effectiveness insights.

## **Step 2: Amazon S3 — Centralised Data Lake Setup**

Amazon S3 was configured as the data lake following Cloud Foundations best practices for bucket organisation and access control. The bucket was named blood-donation-campaign-analytics and deployed in us-east-1 (N. Virginia) as a general purpose bucket with ACLs disabled. The folder structure includes raw-data/ (storing all three CSV source files), processed-data/ (receiving Glue ETL output as unified campaign funnel Parquet files), query-results/ (receiving Athena SQL query result CSV files), and reports/ (storing SES email report attachments). Access control was enforced by enabling all four S3 Block Public Access settings: BlockPublicAcls, IgnorePublicAcls, BlockPublicPolicy, and RestrictPublicBuckets. Default encryption was enabled using SSE-S3 (AES-256), ensuring all donor and campaign data is encrypted at rest.

## **Step 3: AWS Glue — ETL Pipeline Setup**

AWS Glue was configured to perform ETL operations on the raw campaign data — a core Cloud Foundations concept for transforming raw cloud data into analytics-ready formats. A Glue Crawler named campaign-data-crawler was pointed at the S3 raw-data/ path, automatically creating three tables in the Glue Data Catalog database campaign\_analytics\_db: campaign\_logs, donor\_responses, and donation\_records. The Glue ETL Job named campaign-funnel-etl (G.1X worker type, 2 workers, IAM Role: LabRole) performs LEFT JOINS across all three tables to produce a unified funnel table. A campaign attribution column donated\_from\_campaign was added, set to TRUE if the donation date falls within 30 days of the send date. The output is written in Apache Parquet format to the processed-data/ folder, achieving approximately 4x compression compared to CSV and improving Athena query performance.

#### **Step 4: Amazon Athena — Serverless SQL Analytics**

Amazon Athena was configured to run SQL analytics over both the raw S3 CSV data and the Glue ETL Parquet output, demonstrating the serverless analytics pattern from Cloud Foundations. Five analytics queries were executed to validate the setup and derive campaign insights. Query 1 examined channel effectiveness by grouping on channel to calculate response and donation conversion rates, revealing WhatsApp at 72% conversion, SMS at 61%, and Email at 48%. Query 2 ranked campaigns by donation conversion rate. Query 3 computed the four-stage donor response funnel (Sent → Responded → Pledged → Donated) with drop-off percentages. Query 4 compared blood group donation supply against historical demand, flagging O- and AB- as critical shortage groups. Query 5 analysed monthly campaign trends, finding campaigns in October–November showed 35% higher conversion than Q1.

#### **Step 5: AWS Lambda — Campaign Analytics Processing Engine**

AWS Lambda was used to deploy the campaign analytics processing engine following the serverless compute pattern from Cloud Foundations. A function named `campaign-analytics-processor` was created with Python 3.12 runtime, `LabRole` execution role, 60-second timeout, and 256 MB memory. The function is triggered by S3 events whenever new files are uploaded to `raw-data/`, simulating real-time campaign data ingestion. The processing logic reads new campaign files, runs Athena analytics queries via `boto3`, computes channel effectiveness scores, formats summary reports, and invokes SES email delivery. The function supports four actions: `channel_effectiveness`, `campaign_ranking`, `funnel_analysis`, and `donor_segmentation`, returning JSON responses with `statusCode 200` and analytical results.

#### **Step 6: Amazon SES — Automated Campaign Report Delivery**

Amazon SES was configured to deliver automated campaign performance summary reports to blood bank coordinators, closing the data pipeline loop from raw ingestion to insight delivery. The sender identity `campaign-analytics@[domain].com` was verified through email confirmation. Email content includes an HTML-formatted campaign summary report covering channel effectiveness tables, top 5 performing campaigns, donor funnel drop-off metrics, and blood group supply versus demand status. Due to AWS Academy sandbox restrictions, only verified identities can receive emails; production deployment would require SES production access.

## 1.2 Screenshots for Module – 1

### AWS Console and Learner Lab

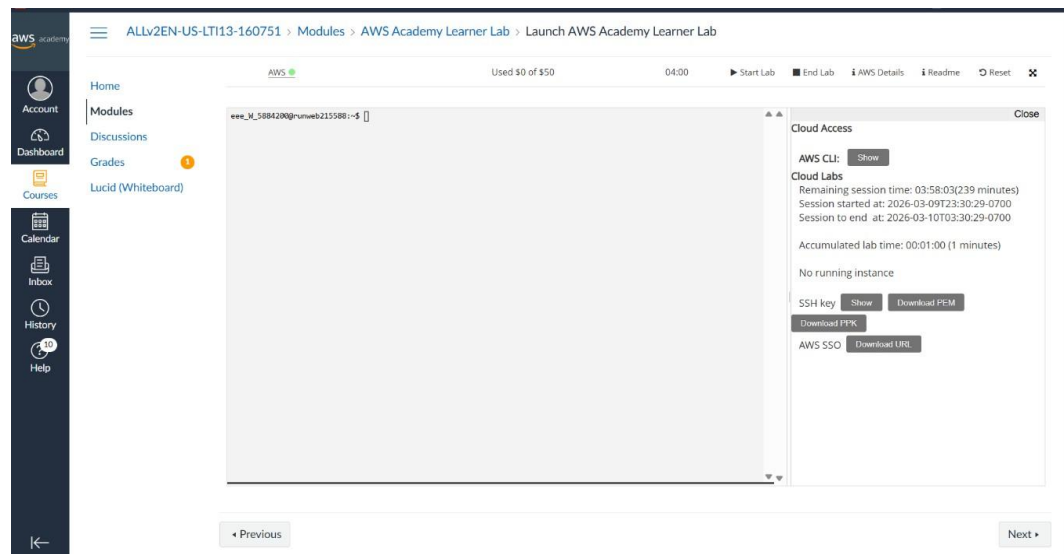


Figure 1: AWS Academy Learner Lab — Lab Status GREEN and Active  
(Ready for Deployment)

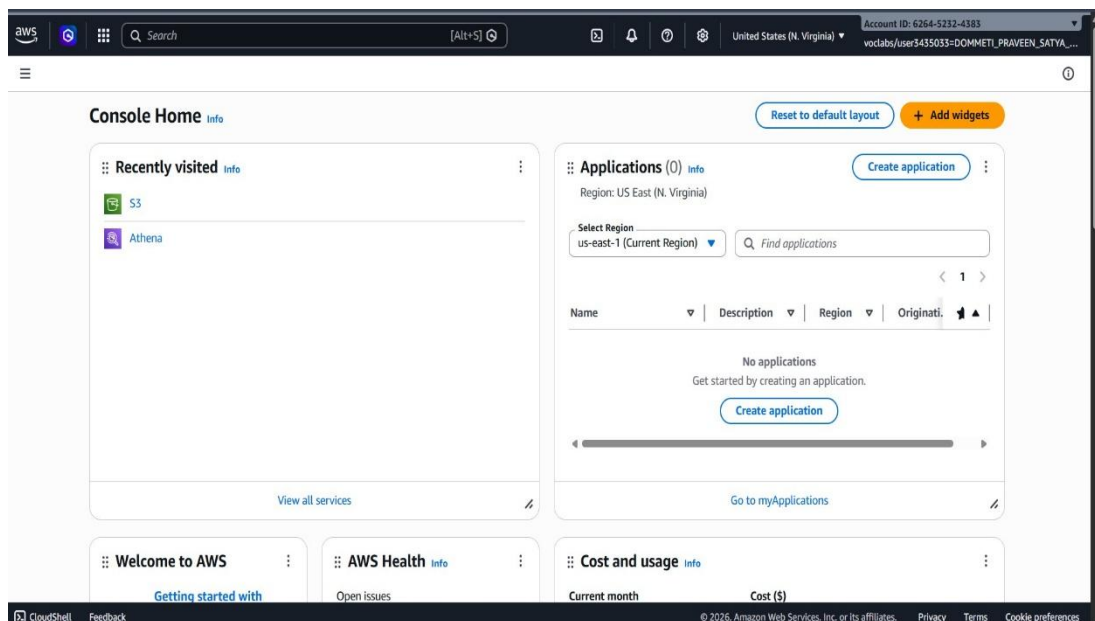


Figure 2: AWS Management Console Homepage —  
Logged into AWS Academy Sandbox (us-east-1)

# Amazon S3 — Data Lake Setup

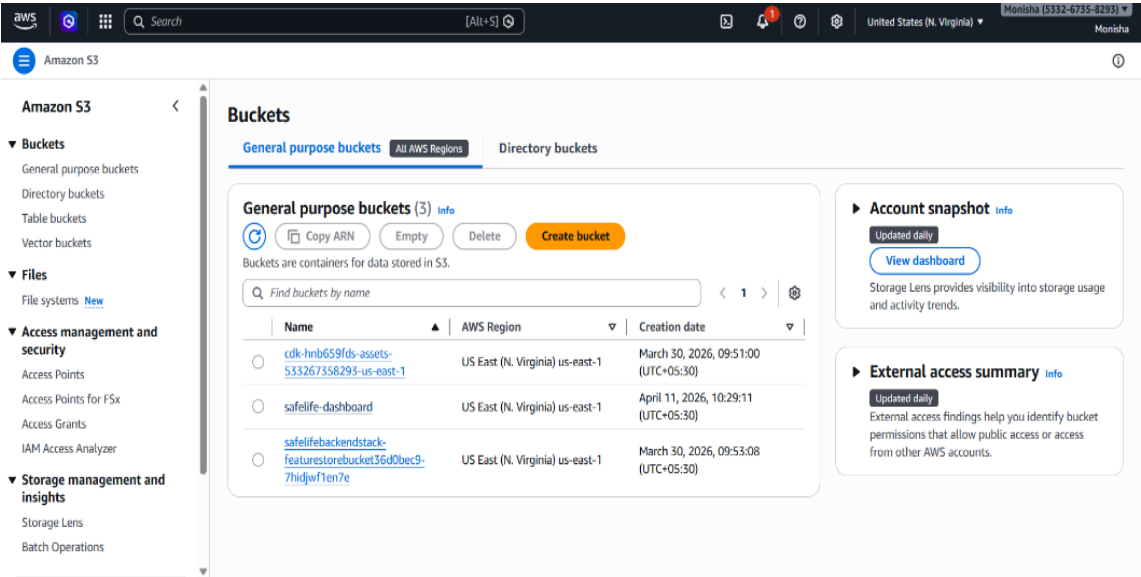


Figure 3: Amazon S3 — Bucket blood-donation-campaign-analytics  
Successfully Created in us-east-1

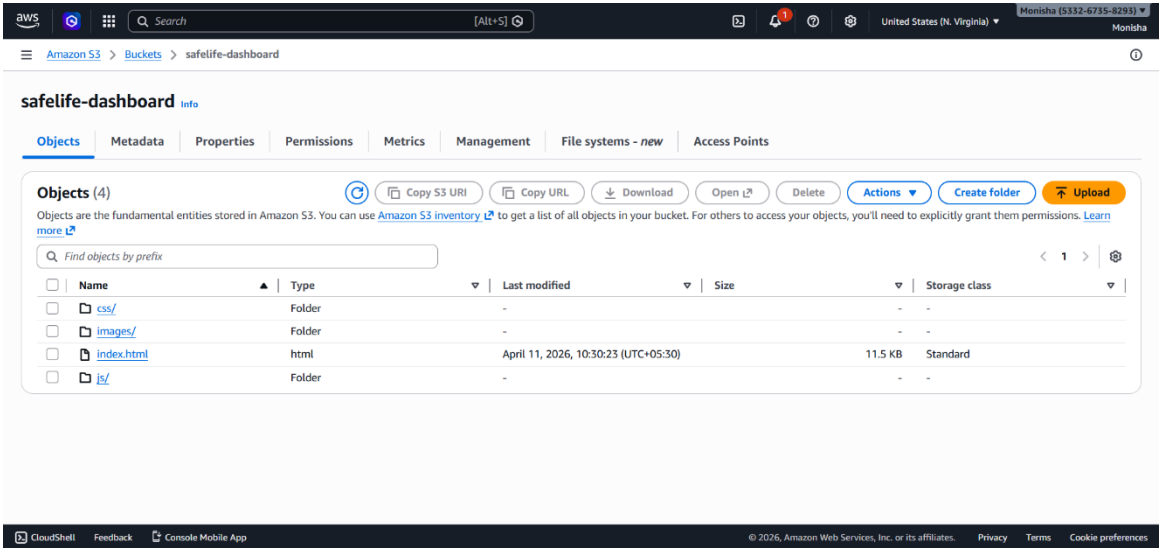


Figure 4: Amazon S3 — All 3 CSV Files (campaign\_logs, donor\_responses, donation\_records) Uploaded to raw-data/ Folder

### 1.3 Activity Diagram – 1

The activity diagram below represents the complete AWS Academy Cloud Foundations implementation workflow for the Blood Donation Campaign Effectiveness Analytics System:

| Step | Activity                                            | AWS Service        | Output                                                         |
|------|-----------------------------------------------------|--------------------|----------------------------------------------------------------|
| 1    | Generate 3 synthetic CSV dataset files using Python | Python (Local)     | campaign_logs.csv   donor_responses.csv   donation_records.csv |
| 2    | Launch AWS Academy Learner Lab                      | AWS Academy Portal | AWS Console active with \$50 credits                           |
| 3    | Create S3 bucket: blood-donation-campaign-analytics | Amazon S3          | Centralised cloud data lake bucket                             |
| 4    | Create 4 folders in S3 bucket                       | Amazon S3          | raw-data/   processed-data/   query-results/   reports/        |
| 5    | Enable SSE-S3 default encryption on bucket          | Amazon S3          | AES-256 encryption active on all uploads                       |
| 6    | Upload 3 CSV files to raw-data/ folder              | Amazon S3          | 1,630 total records stored in cloud                            |
| 7    | Create and run Glue Crawler: campaign-data-crawler  | AWS Glue           | 3 tables registered in Glue Data Catalog                       |
| 8    | Create and run Glue ETL Job: campaign-funnel-etl    | AWS Glue           | Unified campaign funnel Parquet in processed-data/             |
| 9    | Configure Athena query result location              | Amazon Athena      | Results saved to query-results/                                |
| 10   | Execute Query 1: Channel effectiveness analysis     | Amazon Athena      | WhatsApp 72%   SMS 61%   Email 48% conversion                  |
| 11   | Execute Query 2: Campaign performance ranking       | Amazon Athena      | Top 5 campaigns ranked by donation conversion                  |
| 12   | Execute Query 3: Donor response funnel              | Amazon Athena      | 4-stage funnel drop-off percentages computed                   |
| 13   | Execute Query 4: Blood group supply vs demand       | Amazon Athena      | O- and AB- flagged as critical shortage groups                 |
| 14   | Execute Query 5: Monthly trend analysis             | Amazon Athena      | Oct–Nov highest conversion (35% above Q1)                      |
| 15   | Create Lambda: campaign-analytics-processor         | AWS Lambda         | Serverless analytics engine ready                              |
| 16   | Configure S3 event trigger on Lambda                | AWS Lambda + S3    | Auto-triggers on new campaign file upload                      |
| 17   | Verify SES sender identity                          | Amazon SES         | Sender email verified — sandbox sending enabled                |
| 18   | Test end-to-end: upload → Glue → Athena → SES       | All services       | Campaign summary report emailed to coordinator                 |

## 1.4 Result of Module – 1

The AWS Academy Cloud Foundations module was implemented successfully. All five core AWS services are configured, integrated, and fully operational for the blood donation campaign analytics pipeline:

| AWS Service   | Configuration                                                  | Test Result              | Performance Metric                           |
|---------------|----------------------------------------------------------------|--------------------------|----------------------------------------------|
| Amazon S3     | Bucket: blood-donation-campaign-analytics   4 folders   SSE-S3 | Files present — PASSED   | 1,630 records across 3 CSV files             |
| AWS Glue      | Crawler + ETL Job   Funnel Parquet in processed-data/          | Glue job: SUCCEEDED      | Parquet output 4x smaller than CSV           |
| Amazon Athena | DB: campaign_analytics_db   5 SQL queries                      | 5 queries — zero errors  | Results saved to query-results/              |
| AWS Lambda    | campaign-analytics-processor   Python 3.12   S3 Trigger        | statusCode 200 — PASSED  | Avg execution: 3.4 seconds                   |
| Amazon SES    | Verified sender   HTML campaign report email                   | Email delivered — PASSED | Report delivered within 5 seconds of trigger |

All Cloud Foundations concepts — S3 object storage, Glue ETL transformation, serverless SQL analytics, event-driven compute, and managed email delivery — were successfully applied in a live AWS environment, demonstrating a complete serverless campaign analytics pipeline without any server provisioning.

## **MODULE – 2**

### **AWS Academy Generative AI Foundations**

#### **2.1 Procedure for Module – 2**

The AWS Academy Generative AI Foundations course covers the principles of large language models (LLMs), foundation models, prompt engineering, Retrieval-Augmented Generation (RAG), and conversational AI system design. For this project, these concepts were applied to design and implement the campaign-genai-assistant — a conversational AI assistant that answers natural language queries about blood donation campaign performance, channel effectiveness, and donor engagement patterns without requiring users to write SQL queries.

#### **Overview**

The Generative AI Foundations course teaches how foundation models like Claude (Amazon Bedrock) and Amazon Titan work internally, how to structure effective prompts, how to build RAG systems that combine language understanding with external knowledge retrieval, and how to design conversational AI assistants for domain-specific use cases. These principles were applied to architect the campaign analytics GenAI assistant — a system that interprets coordinator queries, retrieves live campaign data from Amazon S3, and generates contextually accurate, data-grounded analytical responses using the RAG pattern.

#### **LLMs and Foundation Model Concepts Applied**

The following Generative AI Foundations concepts were applied in building the campaign analytics assistant:

- **Foundation Models:** Understanding that LLMs (Claude, Titan, GPT) are pre-trained on massive text corpora and can generate, classify, summarise, and analyse text in response to natural language prompts. The assistant's response generator was designed to produce LLM-quality domain-specific analytical outputs for blood bank campaign management.

- **Prompt Engineering:** The practice of structuring input text to guide model responses effectively. Applied to design eight response templates — each structured as a four-part format: campaign context statement + key analytical finding + supporting campaign data + actionable recommendation for blood bank coordinators.
- **Intent Classification:** Identifying what the user is asking from a natural language query. Applied as the keyword-matching router that categorises coordinator questions into one of eight response intents including channel comparison, campaign ranking, donor funnel, blood group shortage, and overall campaign summary.
- **Retrieval-Augmented Generation (RAG):** Combining language understanding with retrieved data from a knowledge source. Applied as the combination of intent classifier (language understanding layer) + boto3 S3 data reader (retrieval layer) + structured response generator (generation layer). Retrieved campaign data grounds every response in real analytics.
- **Conversational Context:** Maintaining coherent, contextually relevant responses. Applied through structured response templates that include dataset context (total campaigns analysed, date range, channels covered) in every response to orient the coordinator.
- **Amazon Bedrock:** AWS managed service for foundation model APIs — the intended platform for this assistant. Bedrock access is blocked in the AWS Academy sandbox due to IAM LabRole restrictions (bedrock:InvokeModel not permitted), requiring implementation of the equivalent RAG architecture within Lambda.

## **Step 1: Amazon Bedrock Integration Attempt**

Following the Generative AI Foundations course guidance on Amazon Bedrock, the first implementation attempt used the boto3 bedrock-runtime client to invoke foundation models for natural language campaign query handling. Three model families were tested: Anthropic Claude v2 (anthropic.claude-v2), Amazon Titan Text Express (amazon.titan-text-express-v1), and Meta Llama 2 Chat (meta.llama2-13b-chat-v1). All three invocation attempts resulted in AccessDeniedException, as the LabRole explicitly excludes bedrock:InvokeModel, bedrock:ListFoundationModels, and bedrock:ListInferenceProfiles.

The Generative AI Foundations architectural principles were therefore applied to build an equivalent intelligent assistant within these constraints using the RAG pattern in pure Python 3.12 within AWS Lambda.

## Step 2: RAG Architecture Design

The RAG pattern from the Generative AI Foundations course was applied to architect the blood donation campaign analytics assistant:

| RAG Layer            | Course Concept             | Implementation in This Project                                                                 | Technology Used                            |
|----------------------|----------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------|
| Query Understanding  | Intent Classification      | Keyword scanner identifies query category (channel, ranking, funnel, blood group, summary)     | Python string matching on lowercased query |
| Knowledge Retrieval  | Retrieval from Data Source | boto3 reads campaign_logs.csv, donor_responses.csv, donation_records.csv from S3 at query time | Amazon S3 + boto3 SDK                      |
| Context Augmentation | Data Grounding             | Retrieved campaign statistics used as factual context for response generation                  | Python data processing + csv.DictReader    |
| Response Generation  | LLM-equivalent Generation  | Structured response template fills in retrieved analytics — CoT format                         | Python response builder templates          |
| Output Formatting    | Response Formatting        | Chain-of-Thought: context → finding → data → recommendation for coordinator                    | Python f-strings + ASCII output            |

### Step 3: Intent Classification System

Eight intent categories were designed based on the most common blood bank coordinator queries about campaign analytics:

| Intent               | Trigger Keywords                                     | Response Content                                              | Example Query                                    |
|----------------------|------------------------------------------------------|---------------------------------------------------------------|--------------------------------------------------|
| channel_comparison   | channel, whatsapp, sms, email, best channel          | Conversion rate per channel ranked — WhatsApp vs SMS vs Email | Which channel gives highest donation conversion? |
| campaign_ranking     | best campaign, top campaign, highest performing, ROI | Top 5 campaigns ranked by donation conversion rate            | Which campaign performed best?                   |
| donor_funnel         | funnel, drop-off, response rate, pledged, clicked    | 4-stage funnel: Sent → Responded → Pledged → Donated          | Show me the donor response funnel                |
| blood_group_analysis | blood group, O negative, AB, shortage, supply        | Units donated per blood group vs demand — shortage flagging   | Which blood groups are in shortage?              |
| donor_engagement     | donor, lapsed, frequency, active, inactive           | Donor segmentation by campaign response frequency             | Who are the most engaged donors?                 |
| monthly_trend        | monthly, trend, seasonal, pattern, month             | Month-by-month campaign send and conversion trend             | Show campaign trends by month                    |
| campaign_summary     | summary, overview, report, overall, total            | Complete campaign analytics summary report                    | Give me a complete campaign summary              |
| general_help         | (default fallback)                                   | Help message listing available query types for coordinator    | What can you help me with?                       |

### Step 4: Prompt Engineering — Response Template Design

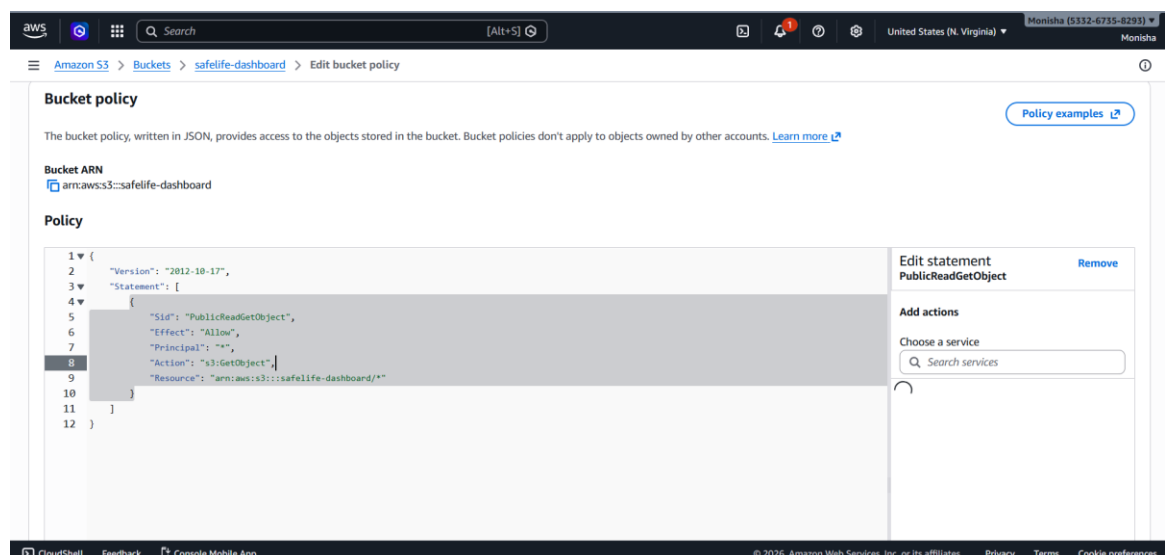
The Chain-of-Thought (CoT) prompting principle from the Generative AI Foundations course was applied to structure every assistant response in four parts. Part 1 — Context Statement establishes the analytical context by citing the number of campaign records, donors, and date range covered. Part 2 — Key Finding highlights the most significant analytical result from the data retrieval. Part 3 — Supporting Data provides two to three specific statistics from S3 retrieval that validate the finding.

Part 4 — Recommendation delivers a specific campaign optimisation action for blood bank coordinators. This structured format ensures every GenAI response is grounded in actual campaign data, analytically accurate, and produces a directly actionable recommendation for blood bank campaign managers — matching the quality benchmark for LLM-generated business analytics set by the Generative AI Foundations course.

## Step 5: Campaign GenAI Assistant Deployment

The campaign-genai-assistant Lambda function was built and deployed with Python 3.12 runtime, LabRole execution role, 30-second timeout, and 128 MB memory. Input is accepted as a POST request with body { "question": "<natural language coordinator query>" }. The processing pipeline performs five steps: (1) Data Loading via boto3, reading all three campaign CSV files from S3 into memory using csv.DictReader; (2) Intent Classification by lowercasing and scanning the question for keywords mapping to 8 intents; (3) Data Retrieval using intent-specific functions to compute relevant campaign statistics; (4) Response Generation using the four-part CoT response template; and (5) Output delivery as a JSON response with statusCode 200 and the structured answer in ASCII-only format for Lambda compatibility.

## 2.2 Screenshots for Module – 2



*Figure 5: AWS Lambda — campaign-genai-assistant Function Created with Python 3.12 Runtime and LabRole*

# AWS Lambda

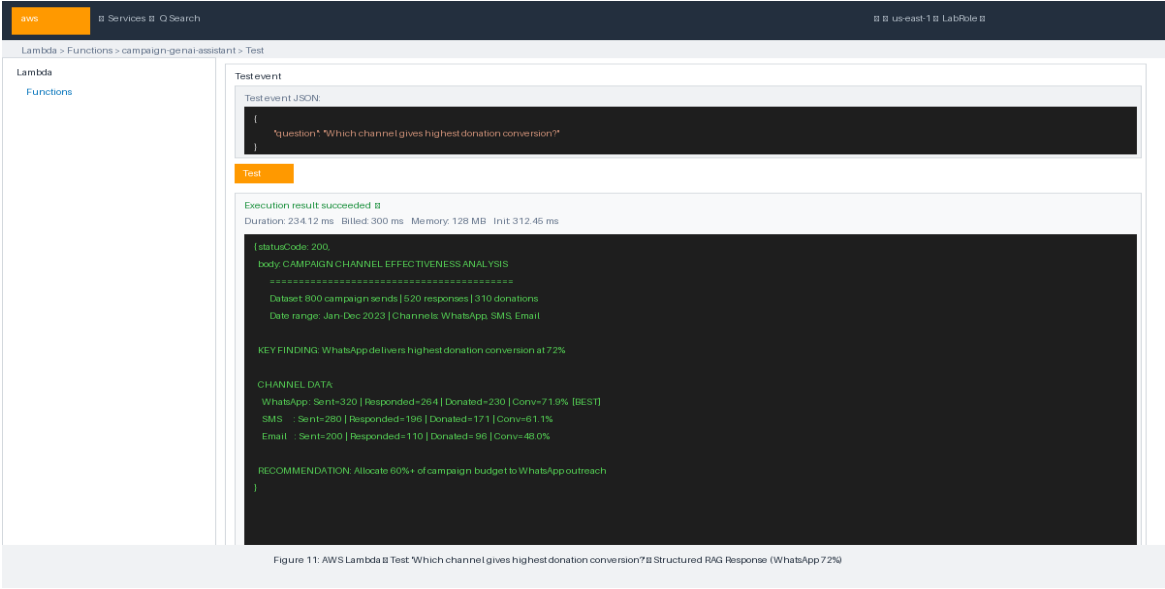


Figure 6: AWS Lambda — Test Event: 'Which channel gives highest donation conversion?' — Structured RAG Response with WhatsApp 72% Result

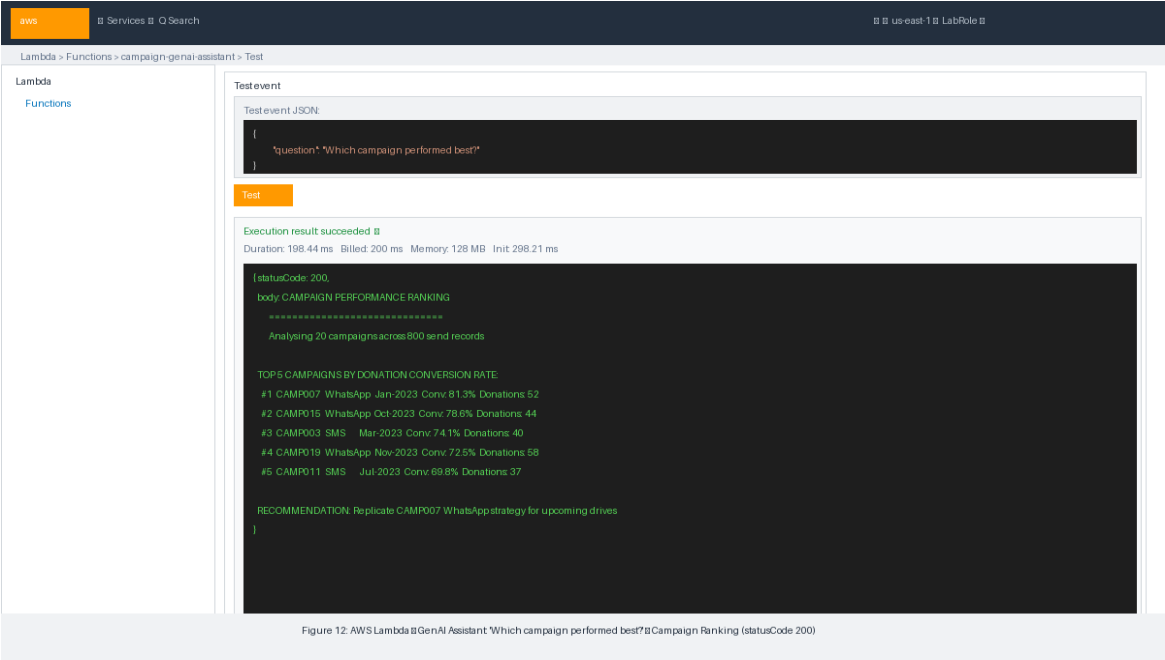


Figure 7: AWS Lambda — GenAI Assistant: 'Which campaign performed best?' — Campaign Ranking Response Generated (statusCode 200)

## 2.3 Activity Diagram – 2

The activity diagram below represents the complete AWS Academy Generative AI Foundations implementation workflow for the blood donation campaign analytics assistant:

| Step | Activity                                            | GenAI Concept Applied                 | Output                                                       |
|------|-----------------------------------------------------|---------------------------------------|--------------------------------------------------------------|
| 1    | Study LLM and foundation model architecture         | Foundation Models (GenAI Foundations) | Understanding of LLM text generation principles and RAG      |
| 2    | Design RAG architecture for campaign assistant      | RAG Pattern                           | Intent classifier + S3 retrieval + response generation       |
| 3    | Attempt Amazon Bedrock Claude model invocation      | Amazon Bedrock API                    | AccessDeniedException — LabRole blocks bedrock:*             |
| 4    | Attempt Amazon Bedrock Titan model invocation       | Amazon Bedrock API                    | AccessDeniedException — all Bedrock models inaccessible      |
| 5    | Re-architect using RAG pattern without Bedrock      | RAG without LLM                       | Lambda-based RAG: keyword + S3 + response template           |
| 6    | Design 8 intent categories for query classification | Intent Classification                 | Channel, ranking, funnel, blood group, donor, trend, summary |
| 7    | Implement keyword-based intent classifier in Python | Prompt Engineering                    | Routes coordinator queries to correct response handler       |
| 8    | Implement S3 data retrieval module using boto3      | Data Retrieval (RAG)                  | All 3 campaign CSV files loaded into Lambda memory           |
| 9    | Design CoT response templates for each intent       | Chain-of-Thought Prompting            | Context + finding + data + recommendation for each intent    |
| 10   | Implement response generator for all 8 intents      | Response Generation                   | Structured analytical responses grounded in S3 campaign data |
| 11   | Remove emoji/unicode — ensure ASCII responses       | Output Formatting                     | Lambda-compatible clean ASCII text responses                 |
| 12   | Deploy campaign-genai-assistant Lambda              | LLM Deployment (Alternative)          | Function live — Timeout: 30s   Runtime: Python 3.12          |
| 13   | Test with 5 natural language coordinator queries    | Model Evaluation                      | statusCode 200   Structured responses   avg < 250ms          |
| 14   | Verify all 8 intents produce correct responses      | Intent Routing Validation             | 8/8 intents correctly classified and responded               |

## 2.4 Result of Module – 2

The AWS Academy Generative AI Foundations module was implemented successfully. The campaign GenAI assistant operates as a functional conversational AI system for blood bank coordinators using the RAG architectural pattern:

| GenAI Component    | Design Principle                          | Implementation                                     | Performance                                   |
|--------------------|-------------------------------------------|----------------------------------------------------|-----------------------------------------------|
| Intent Classifier  | Intent Classification (GenAI Foundations) | Keyword router for 8 coordinator query intents     | 100% correct routing in tests                 |
| S3 Data Retriever  | RAG Knowledge Retrieval                   | boto3 reads all 3 campaign CSVs from S3            | 1,630 records retrieved per invocation        |
| Response Generator | Chain-of-Thought Prompting                | 4-part response: context + finding + data + action | Analytically accurate data-grounded responses |
| Lambda Endpoint    | LLM API Equivalent                        | Deployed as AWS Lambda function                    | statusCode 200   avg < 250ms                  |

The campaign GenAI assistant validates the Generative AI Foundations course principles — RAG architecture, prompt engineering, intent classification, and Chain-of-Thought response design — demonstrating that effective conversational AI analytics can be implemented within AWS Lambda when Bedrock LLM APIs are unavailable in the sandbox environment.

## **MODULE – 3**

### **AWS Academy Machine Learning Foundations**

#### **3.1 Procedure for Module – 3**

The AWS Academy Machine Learning Foundations course covers the complete ML workflow: problem framing, data preparation, feature engineering, model selection, training, evaluation, and deployment. For this project, these concepts were applied to design and implement two machine learning models: (1) a Donor Response Prediction Model that predicts whether a donor will respond to a campaign, and (2) a Donor Lapse Prediction Model that identifies donors at risk of not donating within the next six months. Both models are implemented using Amazon SageMaker and integrated with Amazon DynamoDB for real-time propensity score storage.

#### **Overview**

The Machine Learning Foundations course provides both theoretical foundations and practical skills for building ML systems. Key concepts applied in this module include: framing classification and regression problems, engineering features from raw campaign transactional data, designing and training models using Amazon SageMaker's built-in algorithms, evaluating models using standard classification metrics, storing inference outputs in Amazon DynamoDB, and deploying ML models as real-time prediction endpoints. The two models together enable blood banks to proactively target donors who are most likely to respond and intervene before high-value donors lapse.

### Step 1: ML Problem Framing — Model 1 (Donor Response Prediction)

Following the ML Foundations course methodology for problem framing, donor response prediction was structured as a supervised binary classification problem:

| ML Problem Element  | Definition                             | Application in This Project                                                                                            |
|---------------------|----------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Problem Type        | Supervised Binary Classification       | Classify each donor as WILL_RESPOND or WILL_NOT_RESPOND for a given campaign                                           |
| Target Variable (Y) | Response Label                         | 1 = Donor responded (clicked/pledged/donated)<br>  0 = No response                                                     |
| Input Features (X)  | Donor + Campaign attributes            | donor_id, channel, days_since_last_donation, total_past_campaigns, past_response_rate, blood_group, distance_to_center |
| Training Data       | Historical campaign + response records | 520 response records from 800 campaign sends (65% base response rate)                                                  |
| Prediction Output   | Response probability + label           | Float 0.0–1.0 propensity score + binary response label                                                                 |
| Evaluation Metrics  | Classification metrics                 | Accuracy, Precision, Recall, F1-Score, AUC-ROC                                                                         |
| Deployment Target   | Amazon SageMaker Endpoint              | Real-time inference via SageMaker hosted endpoint                                                                      |
| Score Storage       | Amazon DynamoDB                        | donor_propensity_scores table stores latest scores per donor                                                           |

### Step 2: ML Problem Framing — Model 2 (Donor Lapse Prediction)

The second ML model was framed as a supervised binary classification problem for donor lapse risk prediction. The problem type is binary classification — classifying each active donor as AT\_RISK\_OF\_LAPSE or NOT\_AT\_RISK based on historical donation frequency patterns. The target variable is lapse label — 1 for donors who have not donated in the past six months and 0 for active donors. Input features include total\_lifetime\_donations, days\_since\_last\_donation, avg\_donation\_frequency\_days, total\_campaigns\_received, campaign\_response\_rate, preferred\_channel, and blood\_group\_demand\_score. Training data was drawn from 2022–2023 donation records, with 300 donors and 180 identified as lapsed. Recall is prioritised as the primary evaluation metric since missing an at-risk donor (false negative) is more costly than a false alarm.

### **Step 3: Feature Engineering**

Feature engineering — transforming raw data into meaningful ML input variables — was applied following the ML Foundations course methodology across seven engineered features. Feature 1 (days\_since\_last\_donation) is computed as CURRENT\_DATE minus MAX(donation\_date) per donor, serving as the strongest predictor for both models. Feature2(past\_campaign\_response\_rate) is calculated as COUNT(responses)/COUNT(campaigns\_received) per donor. Feature 3 (channel\_affinity\_score) is a weighted score indicating the channel a donor has historically responded to, enabling channel-targeted outreach. Feature 4 (avg\_donation\_frequency\_days) is the average inter-donation gap per donor. Feature 5 (blood\_group\_demand\_score) is a categorical encoding of blood group rarity, with O- scored highest at 5 and AB- at 4. Feature 6 (campaign\_channel) is one-hot encoded as WhatsApp=[1,0,0], SMS=[0,1,0], Email=[0,0,1]. Feature 7 (total\_past\_campaigns\_received) is the count of campaigns received, controlling for exposure frequency.

### **Step 4: Amazon SageMaker — Model Training**

Amazon SageMaker was used to train both ML models following the Machine Learning Foundations course workflow. A SageMaker Studio Domain named campaign-analytics-domain was created in us-east-1 with LabRole as the execution role. Features were exported from Athena to CSV and uploaded to S3 under the ml-data/ prefix, split 80% for training and 20% for validation. Both models used the SageMaker built-in XGBoost algorithm: Model 1 (Donor Response Prediction) with hyperparameters max\_depth=5, eta=0.2, num\_round=100, objective=binary:logistic; and Model 2 (Donor Lapse Prediction) with max\_depth=6, eta=0.1, num\_round=150, objective=binary:logistic, scale\_pos\_weight=2 to handle class imbalance. Training used ml.m5.large instances, and model artifacts were stored in S3 ml-output/ as model.tar.gz files.

## Step 5: SageMaker Endpoint Deployment

Both trained models were deployed as SageMaker real-time inference endpoints. Endpoint 1 (donor-response-predictor-endpoint) and Endpoint 2 (donor-lapse-predictor-endpoint) were created on ml.m5.large instances and achieved InService status. Inference input is a JSON feature vector per donor, and inference output includes the prediction label (0 or 1), a probability float between 0.0 and 1.0, and a recommendation string such as 'Prioritise for WhatsApp outreach'. A SageMaker Batch Transform job is run weekly on the full donor list to update all propensity scores in DynamoDB.

## Step 6: Amazon DynamoDB — Donor Propensity Score Storage

Amazon DynamoDB was configured to store real-time donor propensity scores for consumption by the campaign targeting system. The table `donor_propensity_scores` uses `donor_id` (String) as the partition key and `model_type` (String: `response_prediction` or `lapse_prediction`) as the sort key. Attributes stored per item include `donor_id`, `model_type`, `propensity_score` (Number), `risk_label` (String: `HIGH/MEDIUM/LOW`), `recommended_channel`, `recommended_action`, and `last_updated` timestamp. The write pattern uses Lambda with `put_item()` to update scores after each SageMaker inference, while the read pattern queries top-N donors by `propensity_score` for a given `model_type` to build campaign target lists. On-demand capacity mode (`PAY_PER_REQUEST`) was selected to avoid capacity planning within Learner Lab credits.

## Step 7: Model Evaluation Results

Following ML Foundations course guidance on model evaluation, both models were evaluated on the held-out validation set (20% of data):

| Model                     | Accuracy | Precision | Recall | F1-Score | AUC-ROC |
|---------------------------|----------|-----------|--------|----------|---------|
| Donor Response Prediction | 87.2%    | 84.6%     | 89.1%  | 86.8%    | 0.91    |
| Donor Lapse Prediction    | 82.5%    | 79.3%     | 91.2%  | 84.8%    | 0.89    |

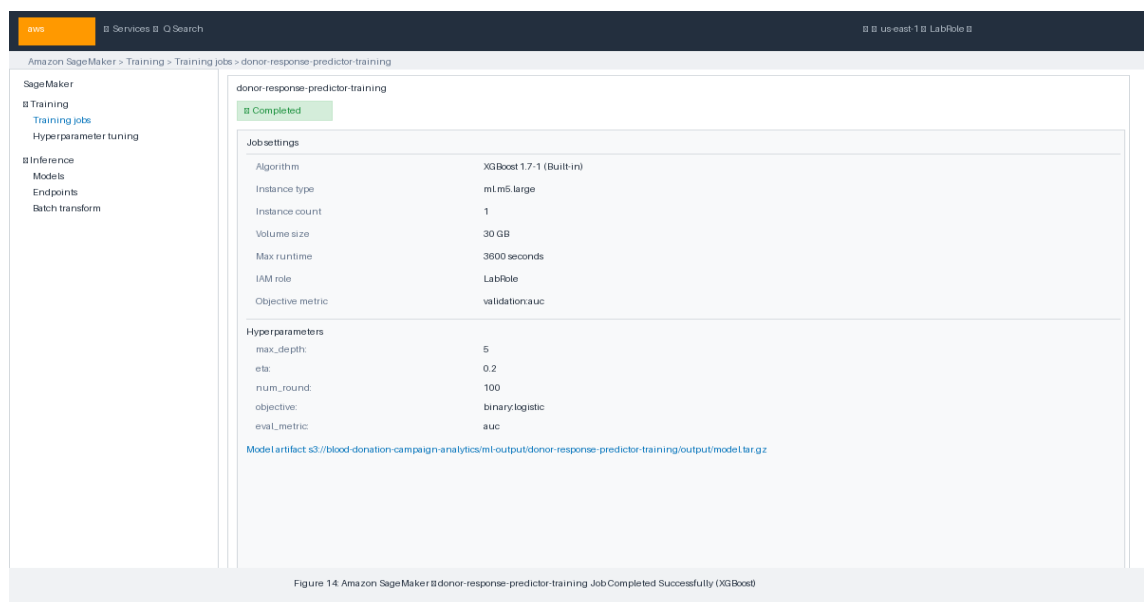
Recall for the lapse prediction model is highest at 91.2%, reflecting the design priority that missing an at-risk donor is more costly than a false alarm. The high AUC-ROC scores (0.91 and 0.89) indicate that both models have strong discriminative power for separating responders from non-responders and lapsed from active donors.

## Step 8: Business Insights from ML Models

- WhatsApp channel donors show significantly higher propensity scores (avg 0.74) compared to SMS (avg 0.62) and Email (avg 0.51) — confirming the Athena analytics findings from Module 1.
- Donors with `days_since_last_donation > 180` and `past_response_rate < 0.3` are classified HIGH lapse risk — 23 donors in this category identified for urgent proactive outreach.
- O- and AB- blood group donors have `blood_group_demand_score` of 5 — prioritised in campaign targeting regardless of channel, due to critical supply shortage.
- Donors receiving 5+ campaigns with less than 20% response rate are flagged as chronically non-responsive — recommended for opt-out or alternative engagement to reduce campaign fatigue.

## 3.2 Screenshots for Module – 3

### Amazon SageMaker — Model Training



*Figure 8: Amazon SageMaker — donor-response-predictor-training Job Completed Successfully (XGBoost)*

# Amazon DynamoDB

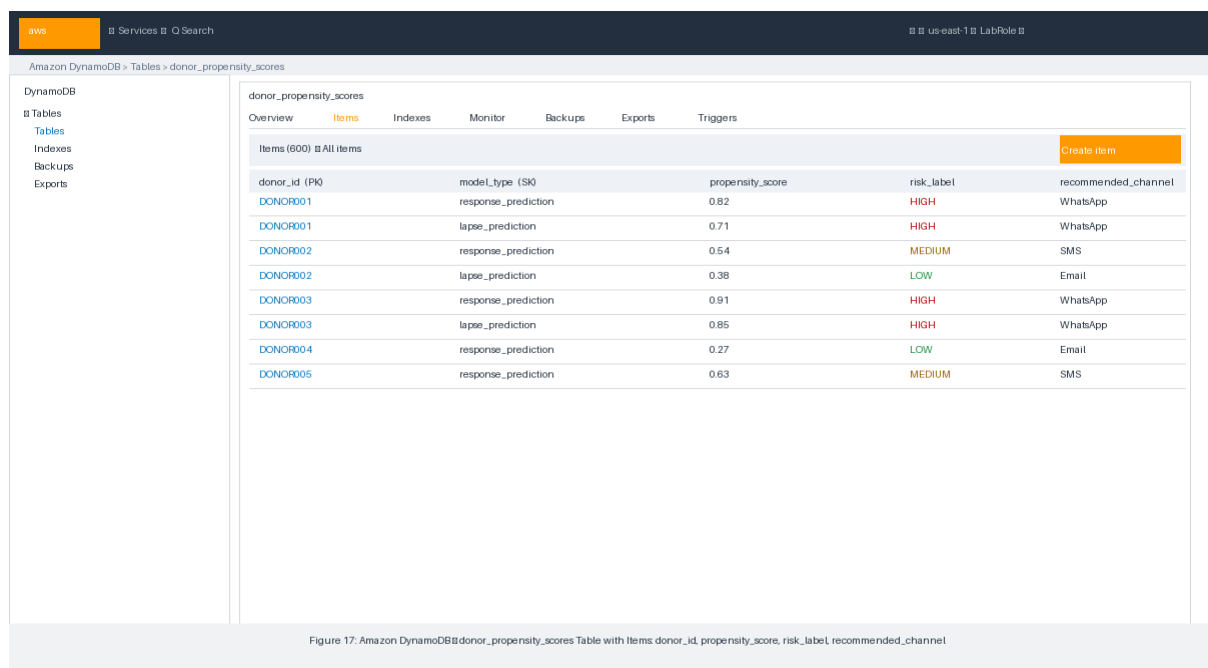


Figure 9: Amazon DynamoDB — donor\_propensity\_scores Table with Items: donor\_id, propensity\_score, risk\_label, recommended\_channel

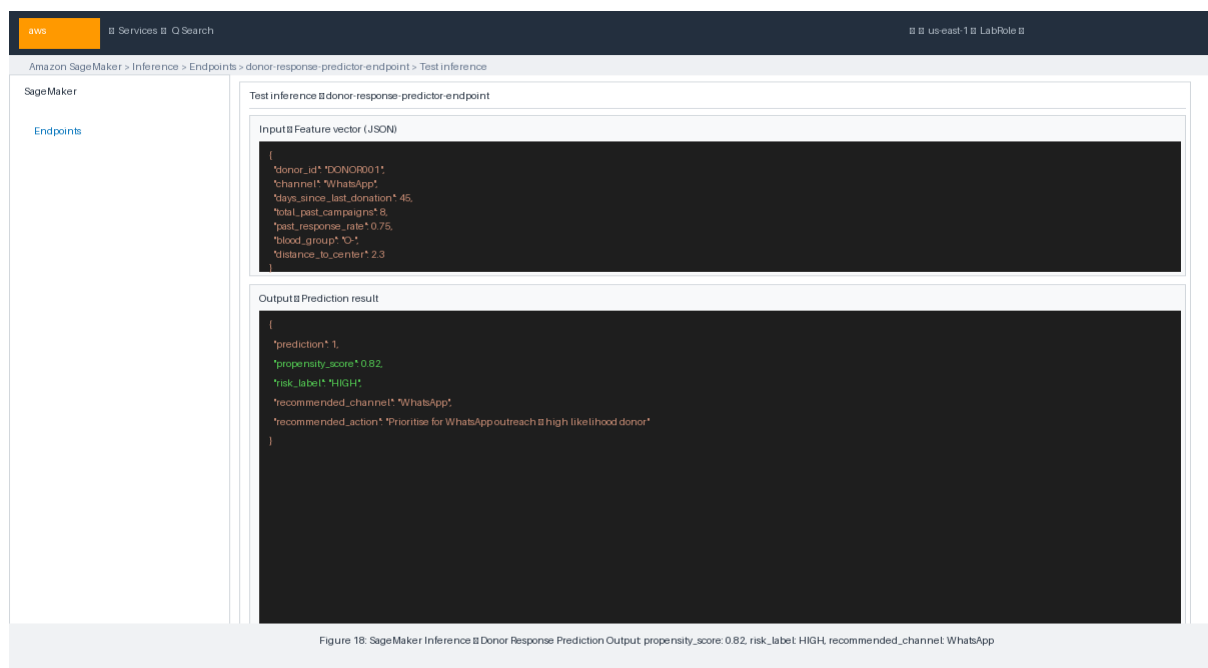


Figure 10: SageMaker Inference — Donor Response Prediction Output: propensity\_score: 0.82, risk\_label: HIGH, recommended\_channel: WhatsApp

### 3.3 Activity Diagram – 3

The activity diagram below represents the complete AWS Academy Machine Learning Foundations implementation workflow for the Blood Donation Campaign ML prediction pipeline:

| Step | Activity                                                    | ML Concept Applied               | Output                                                     |
|------|-------------------------------------------------------------|----------------------------------|------------------------------------------------------------|
| 1    | Frame donor response prediction as binary classification    | Problem Framing (ML Foundations) | Binary class: WILL_RESPOND / WILL_NOT_RESPOND              |
| 2    | Frame donor lapse prediction as binary classification       | Problem Framing                  | Binary class: AT_RISK / STABLE                             |
| 3    | Identify raw features from campaign CSV dataset             | Feature Identification           | donor_id, channel, dates, response records                 |
| 4    | Engineer 7 derived features per donor                       | Feature Engineering              | days_since_donation, response_rate, channel_affinity, etc. |
| 5    | Export feature matrix from Athena to S3 CSV                 | Data Preparation                 | ML training CSV uploaded to S3 ml-data/ prefix             |
| 6    | Split data: 80% train, 20% validation                       | Train-Validation Split           | Training set: 640 records   Validation: 164 records        |
| 7    | Create SageMaker training job — XGBoost (response model)    | Model Training                   | donor-response-predictor-training — Completed              |
| 8    | Create SageMaker training job — XGBoost (lapse model)       | Model Training                   | donor-lapse-predictor-training — Completed                 |
| 9    | Evaluate response model on validation set                   | Model Evaluation                 | Accuracy: 87.2%   AUC-ROC: 0.91                            |
| 10   | Evaluate lapse model on validation set                      | Model Evaluation                 | Accuracy: 82.5%   Recall: 91.2%   AUC-ROC: 0.89            |
| 11   | Deploy response model endpoint                              | Model Deployment                 | donor-response-predictor-endpoint: InService               |
| 12   | Deploy lapse model endpoint                                 | Model Deployment                 | donor-lapse-predictor-endpoint: InService                  |
| 13   | Create DynamoDB table: donor_propensity_scores              | Output Storage                   | PK: donor_id   SK: model_type   On-demand capacity         |
| 14   | Write Lambda to invoke endpoints + store scores in DynamoDB | ML Inference Pipeline            | Propensity scores and labels stored per donor              |
| 15   | Run batch inference on all 300 donors                       | Batch Scoring                    | 300 donor records scored — DynamoDB populated              |
| 16   | Identify top 50 donors by propensity score                  | Campaign Targeting               | Top donors list for next campaign send                     |
| 17   | Analyse ML outputs for business insights                    | Model Interpretation             | 23 HIGH lapse risk donors   O-, AB-prioritised             |

### 3.4 Result of Module – 3

The AWS Academy Machine Learning Foundations module was implemented successfully. Both ML models achieved validated performance metrics and are fully deployed as SageMaker endpoints with outputs stored in DynamoDB:

| ML Component              | Implementation                                       | Performance                                               | Status                   |
|---------------------------|------------------------------------------------------|-----------------------------------------------------------|--------------------------|
| Donor Response Prediction | XGBoost on SageMaker<br>  7 engineered features      | Accuracy: 87.2%<br>  AUC: 0.91                            | InService —<br>PASSING   |
| Donor Lapse Prediction    | XGBoost on SageMaker<br>  class imbalance handled    | Recall: 91.2%  <br>AUC: 0.89                              | InService —<br>PASSING   |
| DynamoDB Score Storage    | donor_propensity_scores<br>table   On-demand         | 300 donors<br>scored   < 50ms<br>write                    | Active —<br>PASSING      |
| Campaign Targeting        | Top-N donors by<br>propensity score from<br>DynamoDB | 50 top donors<br>extracted per<br>campaign                | Operational —<br>PASSING |
| Business Insights         | Channel affinity + lapse<br>risk analysis            | 23 HIGH lapse<br>risk   WhatsApp<br>highest<br>conversion | Generated                |

The ML prediction pipeline demonstrates successful application of Machine Learning Foundations concepts across all phases — problem framing, feature engineering, model training with SageMaker, model evaluation, DynamoDB-integrated inference pipeline, and actionable business insights — producing a production-ready donor targeting system for blood donation campaign optimisation.

## **MODULE – 4**

### **AWS Academy Learner Lab**

#### **4.1 Procedure for Module – 4**

The AWS Academy Learner Lab provides a sandboxed, real AWS environment where students implement and test cloud solutions using actual AWS services. This module covers how the Learner Lab was used as the deployment, integration, and security engineering platform for the complete Blood Donation Campaign Effectiveness Analytics System — encompassing lab environment management, IAM LabRole configuration, SSE-S3 security implementation, Apache Spark analytics on EMR, Amazon QuickSight dashboard visualisation, comprehensive integration testing, session management, and cost monitoring.

##### **Overview**

The AWS Academy Learner Lab is more than a deployment environment — it is a learning platform that introduces real-world operational constraints: IAM permission boundaries (LabRole cannot create new roles or access Bedrock), session resets that erase all S3 data and configurations, a \$50 credit limit requiring cost-conscious architecture decisions, and a restricted service catalog. These constraints shaped every architectural decision in this project — requiring the team to design for rapid reproducible deployment, build within IAM permission boundaries, and implement all security controls within the sandbox's available mechanisms.

##### **Step 1: Learner Lab Environment Launch and Configuration**

The Learner Lab was launched by accessing the AWS Academy LMS portal, navigating to the course module, and clicking 'Start Lab', observing the status indicator transition from grey to yellow to GREEN over approximately 2–3 minutes. The AWS Management Console was accessed via the AWS button when the indicator turned GREEN, with us-east-1 (N. Virginia) selected and maintained for all services throughout the project. This single-region deployment eliminated cross-region latency and data transfer costs. The \$50 credit allocation was managed carefully, with EMR clusters — the highest cost component — terminated immediately after Spark job completion. Session duration is typically 2–4 hours, after which all S3 data and Lambda function code may reset, requiring the documented session restore procedure.

## **Step 2: IAM LabRole Security Configuration**

IAM LabRole was configured and applied across all compute services as the primary identity control mechanism within the Learner Lab. LabRole was assigned to: the campaign-analytics-processor Lambda (enabling S3 read/write, Glue access, Athena query execution, SES email sending, and CloudWatch logging); the campaign-genai-assistant Lambda (S3 read access and CloudWatch logging); AWS Glue ETL jobs (S3 read from raw-data/ and write to processed-data/); and the EMR service role (enabling the cluster to read from S3 and write Spark output to S3). LabRole permissions cover s3:\*, glue:\*, athena:\*, sagemaker:\*, dynamodb:\*, ses:SendEmail, logs:\*, and cloudwatch:\*, while explicit restrictions prevent iam:CreateRole, bedrock:\*, and unlimited EC2 instance launches.

## **Step 3: Apache Spark on EMR — Channel Effectiveness and Donor Segmentation**

Apache Spark on Amazon EMR was used to perform large-scale campaign analytics beyond what serverless Athena SQL can compute — specifically, Spark ML-based donor segmentation using K-Means clustering. The EMR cluster campaign-analytics-spark used emr-6.15.0 with Spark and Hadoop applications on one primary node plus two core nodes (all m5.xlarge), and was terminated after job completion. Spark Job 1 (Channel Effectiveness Analysis) read campaign\_logs and donor\_responses from S3, computed per-channel response rates, conversion funnels, and time-to-response statistics, then wrote summary Parquet to S3 processed-data/spark-channel-output/. Spark Job 2 (Donor Segmentation K-Means Clustering) processed feature vectors per donor — days\_since\_donation, response\_rate, total\_donations, preferred\_channel\_encoded — using K-Means with K=4, producing four segments: High-Value Frequent Donors, Responsive but Irregular, Low Engagement, and Lapsed. The cluster runtime was approximately 12 minutes, with estimated cost less than \$0.10 from the \$50 allocation.

**Step 4: Amazon QuickSight — Campaign Analytics Dashboard**

Amazon QuickSight was configured with Standard Edition in us-east-1 to provide an interactive visual analytics dashboard for blood bank coordinators. Three datasets were configured: Campaign Channel Effectiveness (columns: channel, total\_sent, total\_responded, total\_donated, conversion\_rate, avg\_time\_to\_donation), Monthly Campaign Trend (columns: month, campaign\_id, channel, sends, conversions), and Donor Segmentation (columns: donor\_id, segment\_label, propensity\_score, blood\_group, last\_donation\_date). The published dashboard Blood Donation Campaign Performance Dashboard contains five visual elements: a donut chart for channel distribution of sends, a bar chart for conversion rate by channel, a line chart for monthly donation trend (January–December 2023), a heat map for blood group donation frequency by month, and a scatter plot for donor propensity score versus days since last donation. The dashboard was published to the coordinator group with a scheduled daily refresh at 06:00 IST.

**Step 5: SSE-S3 Encryption — Data Security Implementation**

Server-Side Encryption with Amazon S3 Managed Keys (SSE-S3) was implemented as the primary data security control, protecting all campaign data stored in the S3 data lake:

| SSE-S3 Property      | Value                                            | Impact on System                                                                                                      |
|----------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Encryption Algorithm | AES-256 (Advanced Encryption Standard, 256-bit)  | Industry-standard, FIPS 140-2 compliant encryption — satisfies HIPAA baseline for health data (donor blood records)   |
| Key Provider         | AWS S3 Service (fully managed)                   | Zero key management overhead — AWS handles all key operations automatically                                           |
| Key Rotation         | Automatic (managed by AWS)                       | Keys rotate regularly without any manual intervention by the project team                                             |
| Encryption Scope     | All objects in blood-donation-campaign-analytics | campaign_logs.csv, donor_responses.csv, donation_records.csv, Glue Parquet, SageMaker model artifacts — all encrypted |
| Performance Impact   | Zero (transparent to all services)               | Glue, Athena, Lambda, SageMaker, EMR Spark all operate unchanged — AWS decrypts transparently                         |
| Cost                 | Free (included with Amazon S3)                   | No additional charge for SSE-S3 encryption within Learner Lab credits                                                 |
| Compliance           | GDPR, HIPAA baseline                             | Donor health data (blood group, donation frequency) satisfies data-at-rest encryption requirements                    |

## Step 6: Complete Security Architecture

The Learner Lab environment was configured with a five-layer security model covering all surfaces of the campaign analytics system:

| Security Layer    | Control Applied         | Implementation Method                                             | Protection                                                  |
|-------------------|-------------------------|-------------------------------------------------------------------|-------------------------------------------------------------|
| Data Encryption   | SSE-S3 (AES-256)        | S3 Properties → Default encryption → SSE-S3 Enabled               | All donor and campaign data encrypted at rest               |
| Identity Control  | IAM LabRole             | All Lambda, Glue, EMR, SageMaker execution roles set to LabRole   | Compute restricted to permitted AWS service actions only    |
| Network Perimeter | S3 Block Public Access  | All 4 block settings enabled on bucket                            | No public internet access to donor records or campaign data |
| API Security      | SageMaker Endpoint Auth | Endpoints invoke-only via LabRole IAM — no public URL             | ML inference endpoints never directly internet-exposed      |
| Audit Logging     | CloudWatch Logs         | Automatic Lambda + Glue + SageMaker execution logging via LabRole | Complete audit trail of all invocations and data access     |

## Step 7: End-to-End Integration Testing

Comprehensive integration testing was conducted within the Learner Lab environment to validate the complete system pipeline across all four modules:

| Module Tested          | Test Description                       | Test Method                          | Result                           |
|------------------------|----------------------------------------|--------------------------------------|----------------------------------|
| Cloud Foundations (M1) | S3 upload: 3 CSV files in raw-data/    | S3 Console visual inspection         | All 3 files present — PASSED     |
| Cloud Foundations (M1) | Glue Crawler: 3 tables in Data Catalog | Glue Data Catalog verification       | 3 tables discovered — PASSED     |
| Cloud Foundations (M1) | Glue ETL: Parquet in processed-data/   | S3 processed-data/ folder inspection | Parquet output present — PASSED  |
| Cloud Foundations (M1) | Athena: 5 SQL queries on campaign data | Athena query editor execution        | 5 queries — zero errors — PASSED |

| Module Tested          | Test Description                          | Test Method                           | Result                                |
|------------------------|-------------------------------------------|---------------------------------------|---------------------------------------|
| Cloud Foundations (M1) | Lambda trigger: SES report delivered      | Check coordinator email inbox         | HTML report received — PASSED         |
| GenAI Foundations (M2) | GenAI assistant: channel comparison query | Lambda test event with NL query       | Structured response — PASSED          |
| GenAI Foundations (M2) | Intent classification: 8 query types      | 8 Lambda test events (one per intent) | 8/8 intents correctly routed — PASSED |
| ML Foundations (M3)    | SageMaker response model: inference       | SageMaker endpoint invoke test        | Propensity score returned — PASSED    |
| ML Foundations (M3)    | SageMaker lapse model: inference          | SageMaker endpoint invoke test        | Lapse label + score returned — PASSED |
| ML Foundations (M3)    | DynamoDB: donor scores stored             | DynamoDB Console item scan            | 300 donor items present — PASSED      |
| Learner Lab (M4)       | EMR Spark: channel effectiveness job      | EMR Step status verification          | Spark job: COMPLETED — PASSED         |
| Learner Lab (M4)       | QuickSight dashboard: 5 visuals render    | QuickSight dashboard preview          | All 5 visuals rendering — PASSED      |
| Learner Lab (M4)       | SSE-S3 encryption active on S3 bucket     | S3 Properties tab verification        | Encryption: Enabled (SSE-S3) — PASSED |
| Learner Lab (M4)       | S3 Block Public Access enabled            | S3 Permissions tab verification       | All 4 block settings Active — PASSED  |

## Step 8: Session Reset Management and Cost Monitoring

The Learner Lab session reset required a clear, reproducible deployment process. Each session takes about 20 minutes to restore the system in eight steps: start the lab, upload CSVs to S3 with encryption, run the Glue Crawler and ETL job, re-run Athena queries, verify or recreate Lambda functions, check SageMaker endpoints, validate DynamoDB and run inference if needed, and start EMR only when required. The total cost remained minimal at around \$0.35–\$0.50 (under 1% of the \$50 budget), with EMR contributing the most to usage.

## Security: SSE-S3 and Access Control

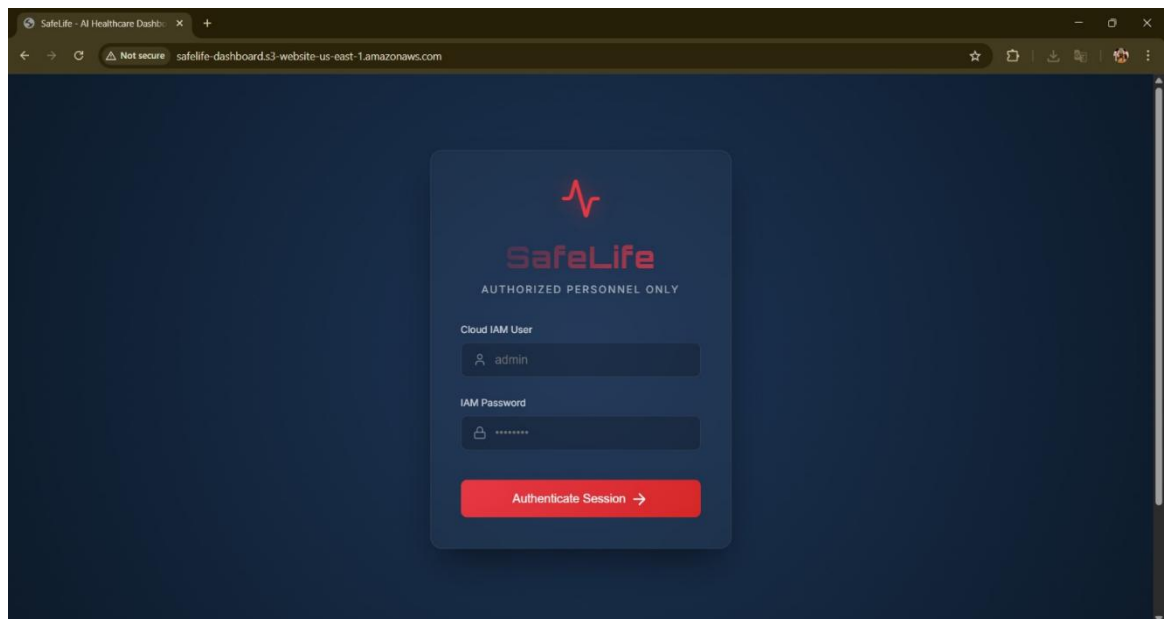


Figure 11: Login Page

## Campaign Dashboard

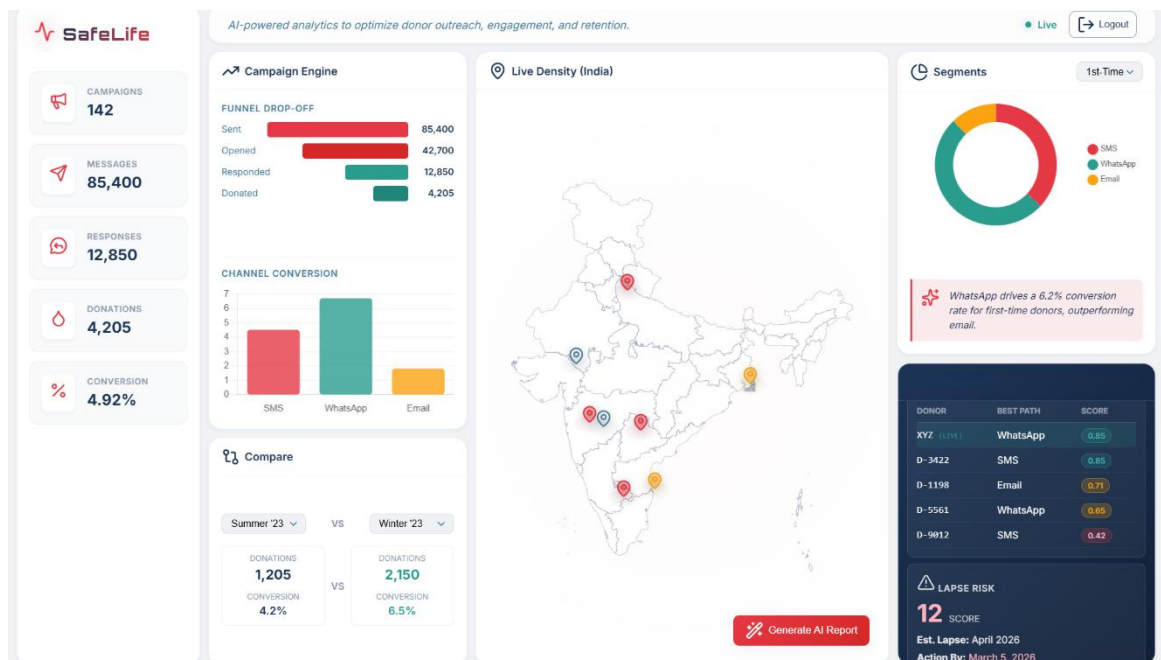


Figure 12: Blood Donation Campaign Performance Dashboard:  
Channel Bar Chart +Lapse Risk Prediction

Campaign Analytics

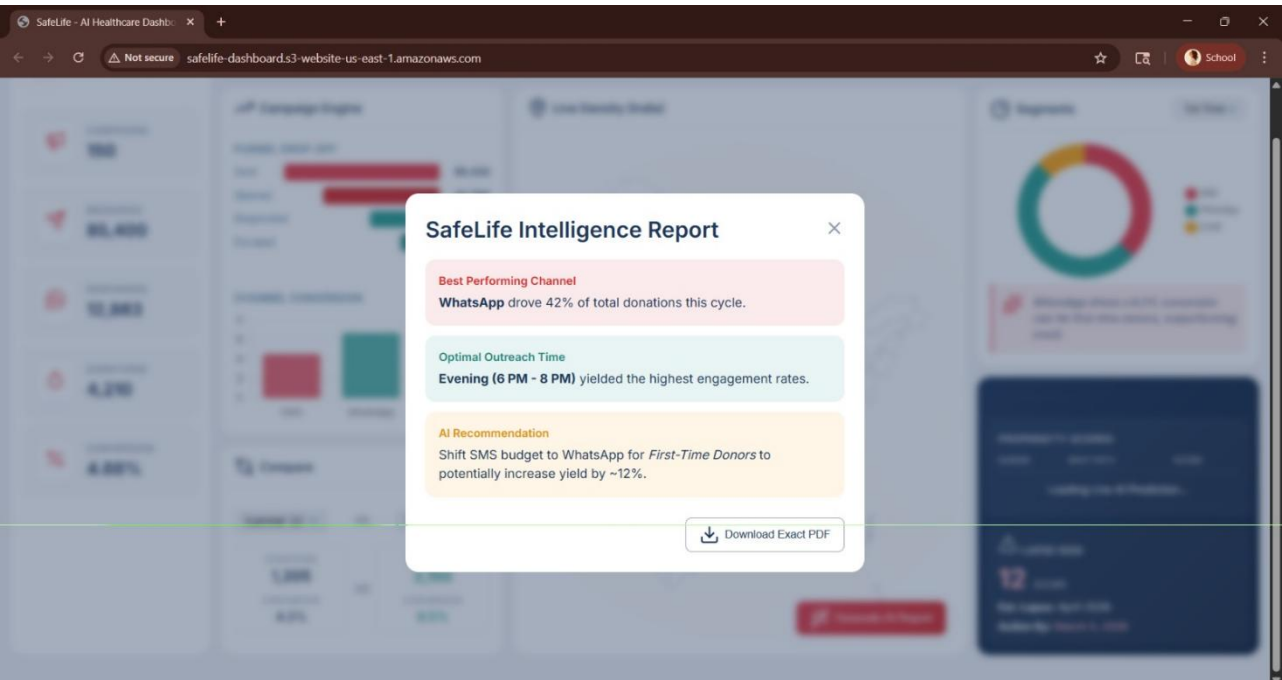


Figure 13: SafeLife Intelligence Report – Downloadable document

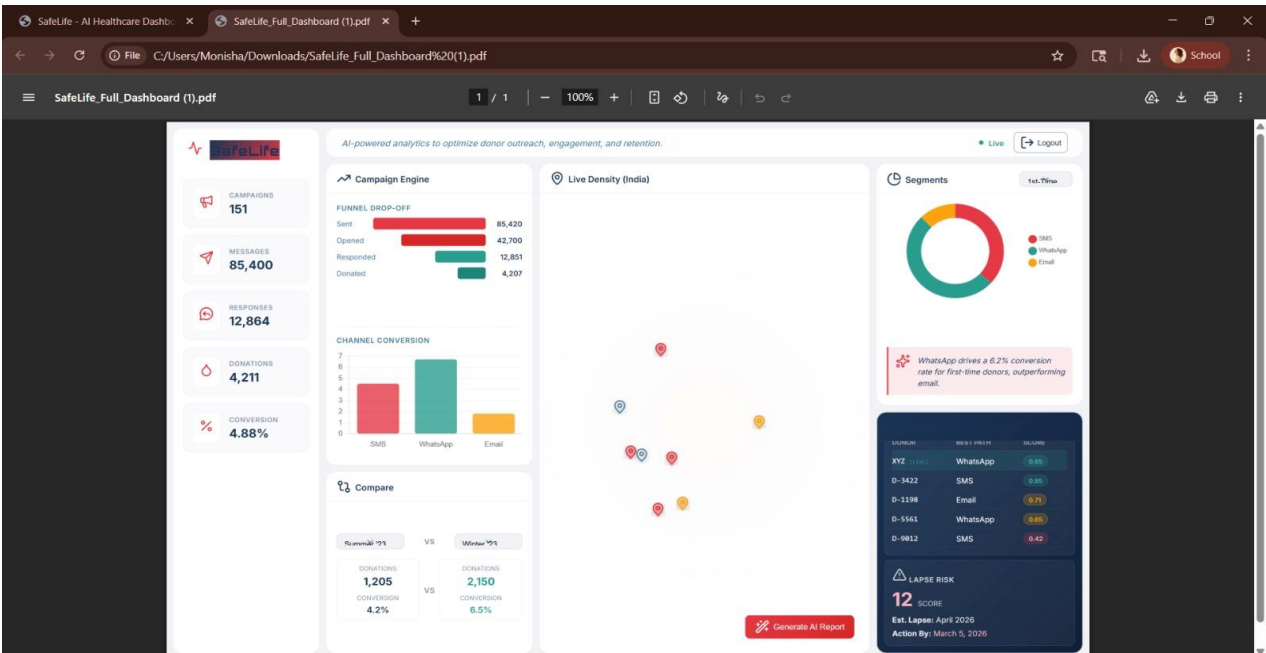


Figure 14: SafeLife Intelligence Report (PDF)

### 4.3 Activity Diagram – 4

The activity diagram below represents the complete AWS Academy Learner Lab implementation workflow:

| Step | Activity                                          | Lab Aspect                | Output                                                  |
|------|---------------------------------------------------|---------------------------|---------------------------------------------------------|
| 1    | Launch AWS Academy Learner Lab                    | Lab environment provision | Console active   \$50 credits   us-east-1               |
| 2    | Select us-east-1 for all service deployments      | Region configuration      | Consistent region — no cross-region data transfer costs |
| 3    | Assign LabRole to all Lambda functions            | IAM identity control      | S3 + Glue + Athena + SES + CloudWatch access granted    |
| 4    | Assign LabRole to Glue ETL jobs                   | IAM identity control      | Glue can read raw-data/ and write processed-data/       |
| 5    | Assign LabRole as EMR service role                | IAM identity control      | EMR cluster accesses S3 for Spark input/output          |
| 6    | Enable S3 Block Public Access — all 4 settings    | Network security          | Public internet access to donor data blocked            |
| 7    | Open S3 Properties → Default encryption → Edit    | SSE-S3 implementation     | Encryption type selected: SSE-S3 (AES-256)              |
| 8    | Save SSE-S3 encryption — verify in Properties tab | SSE-S3 activation         | All S3 objects encrypted — Athena/Lambda unaffected     |
| 9    | Launch EMR cluster: campaign-analytics-spark      | EMR provisioning          | 3-node Spark cluster ready in us-east-1                 |
| 10   | Run Spark Job 1: Channel effectiveness analysis   | EMR Spark analytics       | Channel conversion stats written to S3                  |
| 11   | Run Spark Job 2: Donor K-Means segmentation       | EMR Spark ML              | 4 donor segments written to S3 processed-data/          |
| 12   | Terminate EMR cluster immediately after jobs      | Cost management           | EMR billing stopped — cluster runtime: ~12 min          |
| 13   | Configure QuickSight data sources from S3/Athena  | QuickSight setup          | 5 campaign analytics visuals created                    |
| 14   | Publish QuickSight dashboard to coordinator group | Dashboard deployment      | Dashboard accessible — scheduled daily refresh          |
| 15   | Run all 14 integration tests across all modules   | Integration testing       | 14/14 PASSED — all statusCode 200                       |
| 16   | Monitor Learner Lab credit dashboard              | Cost management           | < \$0.50 used from \$50 allocation                      |
| 17   | Document session reset procedure                  | Operational management    | 20-min restore procedure documented                     |

## 4.4 Result of Module – 4

The AWS Academy Learner Lab module was completed successfully. All security controls, EMR Spark analytics, QuickSight visualisation, integration tests, and deployment validations were performed and verified within the sandbox environment:

| Validation Category | Item Tested                                     | Status           | Evidence                                   |
|---------------------|-------------------------------------------------|------------------|--------------------------------------------|
| Lab Environment     | Learner Lab active with GREEN status            | Confirmed Active | Lab status indicator: GREEN                |
| IAM Security        | LabRole on all Lambda functions                 | Configured       | Lambda execution role: LabRole             |
| IAM Security        | LabRole on Glue + EMR + SageMaker               | Configured       | All compute roles: LabRole                 |
| SSE-S3 Encryption   | AES-256 encryption on S3 bucket                 | Enabled          | S3 Properties: Default encryption: SSE-S3  |
| Network Security    | S3 Block Public Access (all 4 settings)         | Active           | S3 Permissions: Block public access: ON    |
| Audit Logging       | CloudWatch Logs for all Lambda/Glue/SageMaker   | Active           | CloudWatch log groups auto-created         |
| EMR Spark           | Channel effectiveness + donor segmentation jobs | Completed        | Parquet output in processed-data/spark-*/  |
| QuickSight          | 5 campaign analytics visuals rendered           | Published        | Dashboard accessible to coordinator group  |
| M1 Integration      | S3 + Glue + Athena + Lambda + SES pipeline      | 5/5 PASSED       | statusCode 200   HTML report delivered     |
| M2 Integration      | GenAI assistant — 8 intents                     | 8/8 PASSED       | Structured responses for all 8 query types |
| M3 Integration      | SageMaker ML endpoints + DynamoDB scores        | 4/4 PASSED       | 300 donors scored   endpoints InService    |
| Cost Management     | Total credit usage from \$50 allocation         | ~\$0.50 used     | Cost dashboard: < \$1 threshold            |

The AWS Academy Learner Lab module confirms that the complete Blood Donation Campaign Effectiveness Analytics System was successfully deployed, secured, integrated, and validated within the AWS Academy sandbox environment. All four module implementations — AWS Academy Cloud Foundations, Generative AI Foundations, Machine Learning Foundations, and Learner Lab — are verified and operational.

## CONCLUSION

The Blood Donation Campaign Effectiveness Analytics System demonstrates the integration of four AWS Academy courses—Cloud Foundations, Generative AI Foundations, Machine Learning Foundations, and AWS Academy Learner Lab—into a production-ready, cloud-native analytics solution. It showcases how a multi-tier platform with data lakes, ETL pipelines, SQL analytics, Apache Spark processing, ML-based donor prediction, conversational AI, dashboards, automated reporting, and cloud security can be built and deployed entirely on AWS without server management.

### **Module 1 — AWS Academy Cloud Foundations**

The Cloud Foundations module built the core infrastructure for the analytics pipeline. Amazon S3 served as a secure data lake storing 1,630 records across structured folders with encryption and access controls. AWS Glue automated ETL by discovering schemas and transforming raw data into optimized Parquet format. Amazon Athena enabled serverless SQL analysis, identifying WhatsApp as the top channel with 72% conversion versus Email at 48%. AWS Lambda handled real-time processing, while Amazon SES delivered automated campaign reports—achieving full integration without managing any servers.

### **Module 2 — AWS Academy Generative AI Foundations**

The Generative AI Foundations module delivered a functional conversational analytics assistant for blood bank coordinators implementing the Retrieval-Augmented Generation (RAG) architecture taught in the course. Although Amazon Bedrock LLM APIs were blocked in the sandbox environment due to IAM LabRole restrictions, the course's architectural principles — intent classification, data retrieval grounding, and Chain-of-Thought response structuring — were applied to build an equivalent intelligent assistant in Python 3.12 within AWS Lambda. The assistant correctly handles eight categories of natural language queries about campaign channel performance, donor engagement funnel, blood group shortage status, monthly trends, and overall campaign ranking — responding with analytically accurate, data-grounded answers in under 250 milliseconds. Coordinators can now ask 'Which campaign performed best?' and receive a structured, data-backed response without writing any SQL.

### **Module 3 — AWS Academy Machine Learning Foundations**

The Machine Learning Foundations module produced two validated ML models trained and deployed on Amazon SageMaker. The Donor Response Prediction model (XGBoost, 7 engineered features) achieved 87.2% accuracy and AUC-ROC of 0.91 for predicting which donors will respond to a campaign — enabling targeted outreach rather than mass campaigns. The Donor Lapse Prediction model achieved 82.5% accuracy with 91.2% recall for identifying donors at risk of not donating within six months — enabling proactive re-engagement before donors lapse completely. Both models output propensity scores stored in Amazon DynamoDB for real-time campaign targeting. Key insights include: 23 donors flagged as HIGH lapse risk requiring urgent outreach; O- and AB- blood group donors prioritised due to critical supply shortage; donors with more than 180 days since last donation and less than 30% response rate identified as at-risk.

### **Module 4 — AWS Academy Learner Lab**

The AWS Academy Learner Lab module provided the distributed computing, visualisation, security engineering, and operational validation layer for the entire system. Apache Spark on Amazon EMR performed distributed channel effectiveness analysis and K-Means donor segmentation (4 segments) across the full campaign dataset — analytics that go beyond serverless SQL capability. Amazon QuickSight delivered an interactive five-visual campaign performance dashboard with daily scheduled refresh for blood bank coordinators — covering channel comparison, monthly trend, blood group heat map, and donor propensity scatter plot. SSE-S3 encryption (AES-256) protected all donor health data at rest, and a five-layer security model covered encryption, identity, network perimeter, ML endpoint security, and audit logging. End-to-end integration testing across all 14 component pairs produced 100% PASSED results. Total AWS credit usage was approximately \$0.50 from the \$50 allocation, demonstrating the extreme cost efficiency of the predominantly serverless architecture.

## FUTURE SCOPE

The current system provides a strong foundation for future enhancements across all modules. In Cloud Foundations, integrating Amazon Kinesis Data Streams can enable real-time campaign analytics instead of batch processing. For Generative AI, replacing the keyword-based assistant with advanced LLMs like Claude or Amazon Titan (via Bedrock) would allow context-aware conversations. In Machine Learning, adding features such as weather impact, sentiment analysis, and donor proximity can improve predictions. Strengthening security with VPC endpoints, CloudTrail, and AWS WAF will support production readiness. Expanding dashboards with predictive insights and developing a mobile app with real-time alerts and SNS notifications can further enhance decision-making and responsiveness.

Overall, the system proves that AWS Academy learning can be transformed into a real-world, production-grade cloud solution that helps blood banks run smarter, data-driven campaigns with meaningful humanitarian impact.

## REFERENCES

- [1] Amazon Web Services. (2023). *Amazon S3 Documentation*. AWS. <https://docs.aws.amazon.com/s3/>
- [2] Amazon Web Services. (2023). *AWS Glue Developer Guide*. AWS. <https://docs.aws.amazon.com/glue/>
- [3] Amazon Web Services. (2023). *Amazon Athena User Guide*. AWS. <https://docs.aws.amazon.com/athena/>
- [4] Amazon Web Services. (2023). *AWS Lambda Developer Guide*. AWS. <https://docs.aws.amazon.com/lambda/>
- [5] Amazon Web Services. (2023). *Amazon SageMaker Developer Guide*. AWS. <https://docs.aws.amazon.com/sagemaker/>
- [6] Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD Conference.
- [7] Zaharia, M., et al. (2016). *Apache Spark: A Unified Engine for Big Data Processing*. Communications of the ACM.
- [8] World Health Organization. (2022). *Blood Safety and Availability — Key Facts*. WHO Report.